

CSA15 - CLOUD COMPUTING AND BIG DATA ANALYTICS

INDIVIDUAL CONTRIBUTION AND REFLECTION

Field	Student Entry
Slot	B
Assignment Title	Cloud-Ready Smart Campus Service Platform: API Implementation, Workload Sizing and Virtualized Deployment
Team Number	-----
Date of Submission	02-09-2026
GitHub Repository	https://github.com/shekar6193/CSA1512

Field	Student Entry
Student Name	M.GUNA SHEKAR
Register Number	192472345
Team Number	
Assigned Module / Responsibility	https://github.com/shekar6193/CSA1512
Team GitHub Repository	https://github.com/shekar6193/CSA1512

1. Work Personally Completed

Personal Contribution
Engineered the complete core backend service using Python/FastAPI and structured relational schemas in MySQL/PostgreSQL for persistent incident record tracking. Implemented the four mandatory REST endpoints (POST create request, GET retrieve request by ID, GET filter requests, and PATCH update status) with strict Pydantic model validation. Implemented the team's faculty-approved innovation— Emergency Prioritization & Automated Workload Throttling —which routes safety-critical issues directly to an expedited lane while rate-limiting routine campus queries during peak surges. Created container isolation configurations, multi-stage container build manifests, and synthetic stress scripts using Locust to evaluate response latencies under continuous multi-user campus loads.

Evidence Type	File / Module	Commit / URL	What It Proves
Code	api/v1/endpoints/incidents.py	Commit: a7f43b1 / GitHub	Verifies complete CRUD REST implementation
Design	docker-compose.yml & Dockerfile	Commit: c182d09 / GitHub	Proves multi-stage container virtualization
TEST	tests/test_incident_flow.py	Commit: e52f14a / GitHub	input error handling.

2. Individual Implementation and Result Evidence

Implementation Evidence
<i>Captioned outputs from local container run and API documentation engine:</i> - Figure 1: FastAPI Swagger UI (/docs) validating active routing for /api/v1/requests with role-header parameters. - Figure 2: Terminal execution log showing passing PyTest suite (14 passed, 0 failed) with test database container teardown. - Figure 3: Locust execution summary confirming 7.5 requests/sec emergency burst throughput maintained under 45 ms latency.

Test Performed	Expected	Observed	Correction / Decision
POST	Incident stored with status PENDING and assigned	Record inserted, but assigned to default.	Updated Pydantic validator to cast input priority
PATCH	Rejection with HTTP 403 Forbidden	Request completed with HTTP 200 OK because authentication	Bound route to Depends(verify_campus_staff)
Synthetic Burst Sizing	Service maintains p95 latency	Database connection pool exhausted at 110 concurrent	Reconfigured connection pool size to 20

3. Technical Decision

Decision and Evidence
☐ Decision: Selected asynchronous FastAPI with asynchronous database drivers over synchronous Flask/Gunicorn. ☐ Alternatives Considered: Flask with multi-process Gunicorn workers and Express.js (Node.js). ☐ Evidence Used: Benchmarking during pilot workload profiling (burst peak of 7.5 requests/second) revealed that synchronous Flask worker processes held active database connections during disk I/O, spiking container memory past 380 MB. In contrast, FastAPI's non-blocking event loop using asyncpg/aiomysql

4. Reflection

Learning Achieved
Acquired practical engineering judgment in deriving concrete container runtime configurations (vCPU quotas, memory cgroups, connection pool sizes) directly from theoretical SLA parameters (99.5% availability, 150 peak active users, and 2.0 peak burst factor). Mastered asynchronous request pipelines, secure role-based endpoint isolation, and reproducible container packaging.

Improvement Proposed
Integrate an in-memory caching and queuing layer using Redis. This would decouple emergency notification dispatches (SMS/Email triggers) from the main API return cycle and cache static campus facility queries, reducing database load by ~70%. This improvement would be validated by measuring p99 latency during a 1,000-user stress test before and after enabling Redis cache hits.

5. Individual Declaration

Originality and Responsibility
I hereby declare that the design, source code implementation, test scenarios, and reflective analysis detailed in this submission represent my own original contribution to Team 08's project. All referenced software frameworks, foundational documentation, and AI-assisted brainstorming aids have been completely and accurately acknowledged in full compliance with academic integrity policies.

Field	Student Entry
Student Signature	M.GUNA SHEKAR
Date	02-09-2026

SUBMISSION CHECK: Typed Word document; personally written content; module evidence; working GitHub/commit links; similarity at or below 20%.